



OASIS Security Report

Server-Side Request Forgery Security Analysis

Date: 2026-06-11 10:49:23 — **Model:** mistral:latest

Vulnerability: Server-Side Request Forgery

Project: clean_contracts

Analysis root: `../clean`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **2** file(s) for Server-Side Request Forgery.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2

Critical

(

0

High		0
Medium		2
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.633
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.633

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.633

Source lines: 1-34

► Finding 1.1: Server-Side Request Forgery vulnerability (Medium)

Vulnerable code

```
`msg.sender.call.value(amount)()`
```

Explanation

The contract allows a user to initiate an external call, which can be exploited for Server-Side Request Forgery.

Impact

Potential access to internal services or data theft via the Solidity smart contract

Entry point

`withdraw` function

Execution path

```
'''  
user > withdraw() > msg.sender.call() > external contract  
'''
```

Parameters

amount

Exploitation steps

- Call the `withdraw` function
- Replace the external contract with an attacker-controlled contract

Conditions

An attacker needs to have Ethereum to execute transactions and must be able to interact with the smart contract

Remediation

Validate and sanitize URLs used in external calls, use allowlists, block private IPs and local hostnames

File 2: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.633

Source lines: 1-42

► Finding 2.1: Server-Side Request Forgery vulnerability found (Medium)

Vulnerable code

```
function airDrop() supportsToken hasNoBalance public{
}
```

Explanation

The function 'airDrop' allows user input to call the 'supportsToken' function, which can be manipulated to perform SSRF attacks.

Impact

Potential data theft or access to internal services via SSRF.

Entry point

/airDrop

Execution path

```

'''
+---> airDrop (caller input)
|
|
| supportsToken
v
Bank.supportsToken() (user-controlled argument) ---- SSRF
vulnerability ---
'''

```

HTTP methods

POST

Parameters

function arguments

Exploitation steps

- Call 'airDrop' function with a crafted URL as an argument to 'supportsToken'
- Exploit SSRF vulnerability in the Bank.supportsToken() call

Example payloads

- `http://10.0.0.1/leak`

Conditions

Attacker has the ability to call 'airDrop' function.

Remediation

Validate and sanitize all URL inputs and use allowlists.

Secure example

```
function airDrop() public supportsToken requires (hasNoBalance) {  
    ...  
}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)